Documentación del Proyecto UMUTickets

Francisco Carlos Bernad Guillén, Francisco Pellicer Alarcón

November 26, 2024

1 Documentación del Proyecto

1.1 Estructura del Proyecto

La estructura del proyecto está organizada de manera modular para facilitar el mantenimiento, la extensión y la reutilización del código. Al dividir el sistema en diferentes paquetes, cada uno con un conjunto bien definido de responsabilidades, se consigue que el código sea más comprensible y manejable.

Los diferentes módulos o paquetes que forman el proyecto UMUTickets son los siguientes:

- **Paquete `cliente/`:** Este paquete agrupa las clases `cliente.py`, `persona.py`, `socio.py`, y `tarjetas.py`. La clase `Cliente` es la clase base para `ClienteEmpresarial` y `ClientePersonal`, por lo que todas estas clases se agrupan en el mismo módulo para mantener una relación jerárquica clara. Por otro lado hemos decidido poner la clase `Persona` en otro módulo ya que tanto `ClienteEmpresarial` como `ClientePersonal` la utilizan de manera distinta. Además, la clase `Reserva` depende de `Cliente` y viceversa, lo que genera una relación estrecha entre ambas pudiendo causar importaciones circulares al depender una de otra, lo que justifica su ubicación en el mismo módulo. Por otro lado, la clase `Socio` está justificada en este paquete ya que tiene una estrecha relación con clientes ya que estos son los que pueden convertirse en socios. Y la clase `Persona` es obvia su justificación, puesto que un cliente particular, aparte de tener un id único, es una persona.

- **Paquete `empresas/`:** Contiene clases como `empresa.py`, `organizador.py`, `umuticekts.py`, y `ventas.py`. Este paquete maneja todo lo relacionado con las empresas y su gestión, facilitando su mantenimiento y extensión. De esta manera, hemos englobado aquí todo lo referente a la parte financiera y de organización. Debido a esto hemos incorporado la clase `Estadísticas` en el módulo de `Umutickets` ya que la relación que tienen entre sí es directa y bicondicional ya que para calcular los beneficios es necesario tanto uno como el otro, y así evitamos posibles importaciones circulares durante el código

- **Paquete `enumerados/`:** Agrupa las clases `tipo_deporte.py` y `tipo_evento.py`, que definen valores enumerados utilizados en todo el sistema, como los tipos de deporte y evento.

- **Paquete `eventos/`:** El módulo `evento.py` contiene la clase abstracta `Evento` y sus clases hijas específicas: `EspectaculoAudiovisual`, `EventoDeportivo` y `FeriaEmpresarial`. Estas clases representan diferentes tipos de eventos, lo que justifica su agrupación en este módulo. Hemos decidido poner las tres clases hijas en el mismo módulo que evento, ya que solo se definía el constructor(no había muchas líneas de código) y nos ahorrábamos posibles problemas de importación circular.

- **Paquete `excepciones/`:** Este paquete contiene excepciones personalizadas para manejar errores específicos del sistema. Tener un paquete separado para las excepciones permite centralizar el manejo de errores, hacer el código más limpio y mejorar la depuración en caso de fallos, ya que todas las excepciones están bien definidas y organizadas en un solo lugar. Es por esto que tenemos muchas clases en este módulo destinadas a ese funcionamiento

- **Paquete `interfaces/`:** Contiene las interfaces relacionadas con la gestión y presentación visual, como `gestor_eventos.py` y `interfaz_grafica.py`. Al ser las dos interfaces resulta ser buena práctica en POO ponerlas juntas en el mismo paquete

Esta organización modular permite una mayor **reutilización** del código, facilita la **extensión** del sistema y mantiene las relaciones entre las clases claras y bien estructuradas diferenciando claramente la funcionalidad llevada acabo por cada paquete.

2 Clases

Aquí se detalla el propósito de las clases más importantes, sus relaciones entre ellas y la justificación de las decisiones de diseño:

2.1 Clase Cliente

Propósito: Modela a un cliente en el sistema, gestionando su identificación, tarjeta de crédito, estado de socio, reservas y eventos. Proporciona métodos para gestionar reservas, realizar pagos y calcular precios con descuentos o aumentos dependiendo del tipo de cliente y evento.

Relaciones:

- **Herencia:** No tiene clases hijas directamente en la clase `Cliente`.
- **Composición:**
  - **Composición con TarjetaPersonal:** Un cliente tiene una relación de composición con `TarjetaPersonal`, ya que cada cliente tiene una tarjeta de crédito asociada.
  - **Composición con TarjetaSocio:** Un cliente puede ser socio o no. Si este decide serlo, se generá automáticamente una `TarjetaSocio` automática y única en el sistema ofreciendo privilegos a dichos clientes.
  - **Composición con Reserva:** Un cliente puede tener varias reservas, lo que indica que la clase `Reserva` también está asociada a `Cliente`.
- **Visibilidad:** Los atributos clave, como `_id_cliente`, `_tarjeta_credito`, `_es_socio`, `_eventos` y `_reservas`, son protegidos, lo que asegura un encapsulamiento adecuado. Para acceder a ellos, se proporcionan métodos getter y setter.

2.2 Clase ClienteParticular

Propósito: Representa a un cliente particular, que es una persona física que utiliza el sistema. Hereda de `Persona` y `Cliente`, lo que le permite gestionar tanto los datos personales (como nombre, apellidos, etc.) como los datos específicos de un cliente (como tarjeta de crédito, reservas y eventos).

Relaciones:

- **Herencia:** Hereda de las clases `Persona` (para gestionar datos personales) y `Cliente` (para gestionar la relación como cliente).
- **Composición:** Composición con `TarjetaPersonal`, gestionando el uso de una tarjeta de crédito asociada al cliente particular.
- **Visibilidad:** La visibilidad de los atributos sigue la convención de la clase base `Cliente`, es decir, los atributos están encapsulados y se accede a través de los métodos correspondientes.

2.3 Clase ClienteEmpresarial

Propósito: Modela a un cliente empresarial que está asociado a una empresa y tiene un contacto específico dentro de la misma. Hereda de la clase `Cliente` y agrega atributos adicionales relacionados con la empresa.

Relaciones:

- **Herencia:** Hereda de `Cliente`. Un `ClienteEmpresarial` es una especialización de `Cliente`, que añade detalles adicionales específicos para clientes empresariales.
- **Composición:** La clase tiene relaciones de composición con `Empresa` y `Persona` (persona de contacto), ya que un cliente empresarial está asociado a una empresa y tiene un contacto en la empresa.
- **Visibilidad:** Los atributos como `empresa` y `persona_contacto` están encapsulados, protegiendo la información sensible relacionada con el cliente empresarial.

Delegación:

- La clase `Cliente` delega las operaciones de pagos y gestión de reservas a las clases correspondientes (`Reserva` y `TarjetaPersonal`).
- La clase `ClienteEmpresarial` y `ClienteParticular` delegan gran parte de la lógica de cliente en la clase base `Cliente`, sin necesidad de implementar esa lógica nuevamente, lo que favorece la reutilización de código y la extensión del sistema.

2.4 Clase Reserva

Propósito: La clase `Reserva` gestiona la información de una reserva realizada por un cliente para un evento específico. Almacena detalles como el cliente, el evento, la cantidad de entradas, el precio final y la fecha de la reserva. También permite verificar si la reserva ha sido pagada y marcarla como tal.

Relaciones:

- **Herencia:** No tiene clases hijas.
- **Composición:** Está compuesta por una relación con `Cliente` (quien realiza la reserva) y `Evento` (al que se asocia la reserva). Un objeto `Reserva` está vinculado a un objeto `Cliente` y a un objeto `Evento`.
- **Visibilidad:** Los atributos como `_evento`, `_fecha_reserva`, `_cantidad`, `_precio_final`, `_cliente` y `_pagado` son protegidos, y se accede a ellos mediante métodos públicos, garantizando el encapsulamiento.
- **Delegación:** La clase `Reserva` delega la gestión del pago a la clase `Cliente` y la gestión de los detalles del evento a la clase `Evento`. La clase `Reserva` solo se encarga de marcar el estado de la reserva como pagada, pero no maneja la lógica del pago ni los detalles completos del evento, que son delegados a otras clases. Esto permite un diseño modular y la reutilización de código.

2.5 Clase Persona

Propósito: Representa la información básica de una persona, incluyendo su nombre, apellidos, dirección postal y DNI. Sirve como clase base para modelar tanto a clientes particulares como a socios.

Relaciones:

- **Herencia:** No tiene clases hijas directamente, pero es una clase base para otras entidades como `ClienteParticular` y `Socio`.
- **Visibilidad:** Los atributos como `_nombre`, `_apellidos`, `_direccion_postal` y `_dni` son protegidos, y se accede a ellos a través de métodos públicos, garantizando el encapsulamiento.
- **Delegación:** La clase `Persona` no delega funcionalidades en otras clases.

2.6 Clase Socio

Propósito: Modela la condición de un socio en el sistema, incluyendo el número de socio, la tarjeta de socio, las fechas de inicio y expiración de la renovación, así como las ganancias generadas por las cuotas de renovación.

Relaciones:

- **Herencia:** No tiene clases hijas.
- **Composición:** La clase `Socio` tiene una relación de composición con `TarjetaSocio`, ya que un socio tiene una tarjeta asociada que gestiona su saldo y renovación.
- **Visibilidad:** Los atributos como `_numero_socio`, `_tarjeta_socio`,`_fecha_inicio_renovacion`, `_fecha_expiracion_renovacion`, y `_ganancias_por_cuotas` son protegidos, y se accede a ellos mediante métodos getter y setter.
- **Delegación:** La clase `Socio` delega las operaciones de deducción de saldo en la clase `TarjetaSocio`, lo que permite que la lógica de gestión de pagos y saldo sea centralizada en la clase `TarjetaSocio`.

2.7 Clase TarjetaPersonal

Propósito: Representa la tarjeta personal asociada a un cliente, permitiéndole gestionar su saldo, realizar recargas y transferir saldo a la tarjeta de socio. Garantiza que el número de tarjeta sea único y que las operaciones de saldo se realicen correctamente.

Relaciones:

- **Herencia:** No tiene clases hijas.
- **Visibilidad:** Los atributos como `_saldo` y `_numero_tarjeta` son privados. Además, el número de tarjeta se gestiona de forma única mediante un método de clase `_generar_num_tarjeta_unico`, mientras que el saldo se gestiona mediante los métodos de la clase.
- **Delegación:** No delega sobre otros objetos, pero utiliza la `TarjetaSocio` para recargar dinero en esta.

2.8 Clase TarjetaSocio

Propósito: Gestiona el saldo de la tarjeta de un socio, permitiendo recargarla, deducir cantidades y controlar los saldos disponibles. También es responsable de asegurar que no se realicen deducciones por encima del saldo disponible.

Relaciones:

- **Herencia:** No tiene clases hijas.
- **Visibilidad:** El atributo `saldo` es privado y se accede mediante los métodos correspondientes.
- **Delegación:** La clase `TarjetaSocio` no delega en otras clases la lógica de gestión del saldo. Es responsable de realizar todas las operaciones relacionadas con el saldo, como la recarga y la deducción de cantidades.

2.9 Clase Empresa

Propósito: Esta clase representa a una empresa que organiza eventos. Almacena información básica como la razón social, el CIF, el teléfono y la dirección postal de la empresa. Es la clase base para las entidades que gestionan los eventos.

Relaciones:

- **Herencia:** No tiene clases hijas directas. Es la base de la clase `Organizador`.
- **Visibilidad:** Los atributos como `_razon_social`, `_cif`, `_telefono`, `_direccion_postal` son protegidos, accediendo a ellos mediante métodos públicos.
- **Delegación:** La clase `Empresa` no delega funcionalidades a otras clases. Gestiona su propia información de manera autónoma.

2.10 Clase Venta

Propósito: Representa la venta de entradas a un cliente para un evento. Almacena información sobre el cliente, el evento, la cantidad de entradas vendidas y el precio final.

Relaciones:

- **Herencia:** No tiene clases hijas.
- **Visibilidad:** Los atributos como `_cliente`, `_evento`, `_cantidad`, y `_precio_final` son protegidos, accesibles mediante los métodos de la clase.
- **Delegación:** La clase `Venta` no delega ninguna funcionalidad a otras clases. Gestiona la información de la venta de entradas de manera directa.

2.11 Clase Organizador

Propósito: Esta clase representa a un organizador de eventos, que es una empresa encargada de gestionar y organizar diferentes tipos de eventos. Permite la creación, modificación y eliminación de eventos, y gestiona un ID único para cada organizador.

Relaciones:

- **Herencia:** Hereda de `Empresa`, lo que le permite gestionar la información básica de la empresa (razón social, CIF, teléfono, etc.), y hereda de **IGestorEventos** que es una interfaz de la que hablaremos posteiormente.
- **Composición:** La clase `Organizador` tiene una relación de composición con la clase `Evento`, ya que gestiona múltiples eventos.
- **Visibilidad:** Los atributos como `_id`, `_eventos_organizados` son privados y se accede a ellos mediante métodos públicos.
- **Delegación:** La clase `Organizador` delega algunas operaciones como la de modificación de eventos o directamente si creación en la clase `Evento`. De esta manera, se puede modificar el evento sin que la clase `Organizador` tenga que gestionar todos los detalles internamente.

2.12 Clase UMUTickets

Propósito: UMUTickets es la clase principal encargada de gestionar la venta y reserva de entradas para eventos. Su responsabilidad incluye la gestión de clientes, eventos, reservas y ventas. Además, también se encarga de calcular estadísticas sobre beneficios y anulaciones.

Relaciones:

- **Composición:** Contiene instancias de las clases `Cliente`, `Evento`, `Reserva` y `Venta`.
- **Asociación:** Se asocia con la interfaz `IGestorEventos` y debe implementar los métodos definidos en esta interfaz.
- **Herencia:** Hereda de la interfaz `IGestorEventos`, lo que la obliga a implementar métodos para gestionar eventos de forma genérica (crear, modificar, eliminar, etc.).

Visibilidad:

- Atributos como `_clientes`, `_eventos`, `_ventas` y `_reservas` son privados, lo que implica que solo pueden ser accedidos o modificados dentro de la clase.
- Los métodos que gestionan clientes, reservas y ventas son públicos para que se puedan utilizar desde fuera de la clase.

Delegación:

- Delegación de la creación de eventos específicos (como `EventoDeportivo`, `EspectaculoAudiovisual`, `FeriaEmpresarial`) a métodos especializados como `crear_evento`, además de usar delegación de manera idónea a la hora de modificar un evento dado.
- Delegación de la creación de reservas y ventas a métodos como `reservar_entrada` y `vender_entrada`, donde la clase `Cliente` gestiona las transacciones de pago.
- Delegación en la clase cliente en la función de anular un evento. Se accede a todos los clientes registrados, se elimina el evento en el sistema y en la lista de eventos de cada cliente que lo haya pagado (Aquí es cuando se hace delegación). Además se ha tenido en cuenta que cuando anulas un evento y lo has reservado pero aún no lo has pagado, se anula dicha reserva usando también delegación.

2.13 Clase Evento (Clase Abstracta)

Propósito: `Evento` es una clase abstracta que define las propiedades comunes a todos los tipos de eventos (como nombre, descripción, fecha, etc.). Es la base para eventos específicos como `EventoDeportivo`, `EspectaculoAudiovisual` y `FeriaEmpresarial`.

Relaciones:

- **Herencia:** Es la clase base de `EventoDeportivo`, `EspectaculoAudiovisual` y `FeriaEmpresarial`.
- **Composición:** Está asociada con las clases `Venta`, `Reserva` y `UMUTickets`, ya que un evento puede tener múltiples ventas y reservas.

Herencia: Es una clase abstracta porque no tiene sentido instanciarla directamente. Los eventos deben ser de tipos específicos (deportivo, espectáculo, feria), por lo que define una interfaz común, pero delega la implementación detallada a sus clases hijas.

Visibilidad:

- Los atributos y métodos fundamentales de un evento son privados o protegidos.
- Los métodos públicos en esta clase permiten obtener información sobre el evento (como entradas disponibles o precio).

Delegación:

- Delegación de la implementación específica del tipo de evento a las clases hijas (`EventoDeportivo`, `EspectaculoAudiovisual`, `FeriaEmpresarial`).

2.14 Clase EventoDeportivo

Propósito: Representa un evento deportivo en particular. Hereda de `Evento` y agrega propiedades y comportamientos específicos para los eventos deportivos, como el tipo de deporte.

Relaciones:

- **Herencia:** Hereda de `Evento`.
- **Composición:** Un evento deportivo puede tener varias reservas y ventas asociadas.

Visibilidad:

- Los atributos y métodos son privados o protegidos. Los métodos públicos permiten interactuar con las propiedades del evento deportivo.

Delegación:

- Delegación de la lógica específica del evento deportivo (como el tipo de deporte) a los métodos de esta clase.

2.15 Clase EspectaculoAudiovisual

Propósito: Representa un evento de tipo espectáculo audiovisual, como conciertos o representaciones teatrales. Hereda de `Evento` y añade características específicas como el artista principal o la edad mínima para asistir.

Relaciones:

- **Herencia:** Hereda de `Evento`.
- **Composición:** Al igual que los otros tipos de eventos, mantiene relaciones con `Venta` y `Reserva`.

Visibilidad:

- Atributos como el nombre del artista o la edad mínima son privados o protegidos.
- Los métodos públicos permiten obtener y modificar estos atributos.

Delegación:

- Delegación de la implementación del evento específico (como artistas o reglas de edad) a los métodos de esta clase.

2.16 Clase FeriaEmpresarial

Propósito: Representa un evento de tipo feria empresarial, con características específicas de este tipo de evento. Hereda de `Evento`.

Relaciones:

- **Herencia:** Hereda de `Evento`.
- **Composición:** Al igual que otros eventos, puede tener reservas y ventas asociadas.

Visibilidad:

- Atributos privados o protegidos. Métodos públicos permiten interactuar con los detalles del evento.

Delegación:

- Delegación de la implementación específica del evento a los métodos de esta clase.

2.17 Clase Estadisticas

Propósito: La clase `Estadisticas` tiene como objetivo calcular diversos tipos de estadísticas sobre las ventas, beneficios, socios y anulaciones en los eventos gestionados por la clase `UMUTickets`. Proporciona métodos estáticos para realizar estos cálculos sin necesidad de crear una instancia de la clase.

Relaciones:

- **Asociación:** Se asocia con las clases `Venta`, `Cliente`, `UMUTickets`, `Evento`, y sus subclases como `EventoDeportivo`, `EspectaculoAudiovisual` y `FeriaEmpresarial`. El método `calcular_beneficio()` usa las instancias de `Venta` y `Cliente` y se asocia con diferentes tipos de eventos, y accede a todos los eventos creados por `UMUTickets` para calcular los beneficios obtenidos.

- **Herencia:** No hereda de ninguna otra clase, ya que es una clase utilitaria para cálculos estadísticos.

Visibilidad:

- Los métodos de la clase son públicos, ya que permiten realizar los cálculos estadísticos desde fuera de la clase, sin necesidad de crear una instancia de `Estadisticas`.
- No tiene atributos privados o protegidos, ya que los métodos operan directamente sobre las instancias pasadas como parámetros.

Delegación:

- No hay delegación explícita en la clase `Estadisticas`. Todos los cálculos son realizados directamente por los métodos de esta clase, sin delegar responsabilidades a otras clases.

2.18 Clase TipoDeporte (Enum)

Propósito: La clase `TipoDeporte` es una enumeración que define los tipos de deportes disponibles, como `FUTBOL`, `TENIS`, y `BALONCESTO`. Se utiliza para clasificar los deportes en los eventos de tipo deportivo.

Relaciones:

- **Composición:** No tiene relaciones de composición, ya que es una enumeración simple.
- **Asociación:** Se utiliza en las clases que definen un evento deportivo, como `EventoDeportivo`, para especificar el tipo de deporte de un evento.
- **Herencia:** Hereda de `Enum`, lo que le permite ser una enumeración de valores predefinidos.

Visibilidad:

- Al ser una enumeración, sus valores (como `FUTBOL`, `TENIS`, y `BALONCESTO`) son públicos y accesibles desde otras clases.

Delegación:

- No hay delegación en la clase `TipoDeporte`, ya que simplemente define valores constantes para ser utilizados en otras clases.

2.19 Clase TipoEvento (Enum)

Propósito: La clase `TipoEvento` es una enumeración que define los tipos de eventos posibles, como `DEPORTIVO`, `ESPECTACULO`, y `FERIA`. Se utiliza para clasificar los eventos según su tipo.

Relaciones:

- **Composición:** Similar a `TipoDeporte`, no tiene relaciones de composición.
- **Asociación:** Se usa en las clases de eventos (`EventoDeportivo`, `EspectaculoAudiovisual`, `FeriaEmpresarial`) para clasificar el tipo de evento.
- **Herencia:** Hereda de `Enum` para ser una enumeración.

Visibilidad:

- Los valores de la enumeración (`DEPORTIVO`, `ESPECTACULO`, `FERIA`) son públicos y accesibles desde cualquier clase.

Delegación:

- Al igual que `TipoDeporte`, `TipoEvento` no delega ninguna responsabilidad a otras clases, ya que solo define constantes.

2.20 Clases de Excepciones

Propósito: Las clases de excepciones tienen como propósito gestionar errores específicos que pueden ocurrir durante la ejecución del sistema. Cada excepción está diseñada para manejar un tipo de error concreto, garantizando que el sistema pueda reaccionar de manera controlada ante situaciones inesperadas.

Relaciones:

- **Composición:** No tienen relaciones de composición, ya que cada clase es independiente y solo hereda de la clase base `Exception`.
- **Asociación:** Cada clase de excepción está asociada a un tipo específico de error que puede ocurrir dentro del sistema (por ejemplo, saldo insuficiente, evento no encontrado, etc.).
- **Herencia:** Todas las clases heredan de `Exception`, lo que les permite ser utilizadas como excepciones personalizadas en el manejo de errores del sistema.

Visibilidad:

- Las clases de excepciones son públicas y accesibles desde cualquier clase que necesite lanzar o capturar una excepción.
- Los mensajes de error también son públicos, ya que se definen dentro del constructor de cada excepción.

Delegación:

- Las excepciones no delegan ninguna responsabilidad a otras clases, ya que su único propósito es representar un error específico. Sin embargo, pueden ser utilizadas por otras clases para gestionar errores de manera adecuada.

Especificaciones de las Clases de Excepciones:

- `ClienteYaSocio`: Lanzada cuando el número de socio es igual al ID del cliente.
- `ClienteEmpresarialError`: Lanzada cuando un cliente particular intenta acceder a un evento empresarial.
- `SaldoInsuficienteError`: Lanzada cuando no hay suficiente saldo en la tarjeta del cliente.
- `CantidadInvalidaError`: Lanzada cuando se intenta recargar una cantidad no permitida (debe ser 5, 10, 20 o 50 euros).
- `PeriodoRenovacionError`: Lanzada cuando el periodo de renovación es inválido (debe ser mensual, trimestral o anual).
- `EventoNoEncontradoError`: Lanzada cuando un evento no se encuentra.
- `FechaInvalidaError`: Lanzada cuando no se proporciona una fecha válida para un evento.
- `AccederFechas`: Lanzada cuando hay un error al intentar acceder a las fechas de un evento.
- `EstablecerEntradas`: Lanzada cuando hay un problema al establecer las entradas para un evento.
- `DeporteInvalidoError`: Lanzada cuando se proporciona un deporte no válido.
- `MaximoEntradasFeriaError`: Lanzada cuando se excede el máximo de entradas disponibles para ferias empresariales.
- `FechaMinimaReserva`: Lanzada cuando la fecha de reserva es menor a la fecha mínima permitida.
- `ReservaYaPagada`: Lanzada cuando una reserva ya ha sido pagada.
- `RecargaNegativaError`: Lanzada cuando se proporciona un valor negativo para una recarga.
- `NoHaySuficientesEntradasError`: Lanzada cuando no hay suficientes entradas disponibles.
- `CompraFueraDelPlazoError`: Lanzada cuando se intenta realizar una compra fuera del plazo permitido.
- `TipoEventoNoReconocidoError`: Lanzada cuando se proporciona un tipo de evento no reconocido.
- `EventoNoEncontradoErrorOganizador`: Lanzada cuando un evento no se encuentra en los eventos organizados por un organizador específico.
- `ModificacionEventoError`: Lanzada cuando ocurre un error al modificar un evento.
- `ReservaInexistenteError`: Lanzada cuando se intenta gestionar una reserva que no existe.

3 Interfaces

Propósito: Las interfaces definen unas "plantillas" que las clases deben cumplir. En este programa, la única interfaz relevante es `IGestorEventos`, que establece los métodos necesarios para gestionar eventos, como crear, modificar y eliminar eventos. (Se ha creado una interfaz gráfica de la que hablaremos posteriormente). Además se pensó, en crear otra interfaz llamada **IPersistencia**, la cual englobaría todo lo referente al trato de datos, pero fue una idea que surgió después de haber realizado la mayor parte de la app, y se decidió no llevarla a cabo, por simplicidad.

Listado de Interfaces:

- `IGestorEventos`: Define los métodos necesarios para gestionar eventos, incluyendo la creación, modificación y eliminación de los mismos. Las clases que implementen esta interfaz deben proporcionar implementaciones para estos métodos, lo que garantiza una estructura consistente para el manejo de eventos.

Métodos definidos en la interfaz `IGestorEventos`:

- `crear_evento`: Método que debe permitir la creación de eventos de diferentes tipos (deportivos, espectáculos, ferias, etc.) con todos los detalles necesarios, incluyendo nombre, descripción, fecha, entradas, y más.
- `modificar_evento`: Método que debe permitir modificar eventos existentes, cambiando propiedades como la fecha, dirección, URL, etc.
- `eliminar_evento`: Método que debe eliminar un evento dado su ID.

Clases que implementan la interfaz `IGestorEventos`:

- `UMUTickets`: La clase `UMUTickets` implementa la interfaz `IGestorEventos`, proporcionando la lógica para crear, modificar y eliminar eventos en el sistema. Esta clase gestiona el ciclo de vida completo de los eventos, desde su creación hasta su eliminación.
- `Organizador`: La clase `Organizador` también implementa la interfaz `IGestorEventos`, y se encarga de la gestión de los eventos desde la perspectiva del organizador. Proporciona las funcionalidades necesarias para que un organizador cree, modifique o elimine eventos. Lo diferente en cuanto a implementacion respecto a UMUTickets es que los organizadores solo pueden modificar eventos que ellos han creado y no otros, en cambio UMUTickets tiene acceso a todos los eventos

4 Gestión de Entrada/Salida en el Proyecto

En este proyecto, la gestión de almacenamiento y recuperación de información se resuelve utilizando archivos JSON. Los datos de los clientes, como su nombre, apellidos, dirección, DNI, y otros atributos, se almacenan en un archivo JSON ubicado en la carpeta `DB` del proyecto. Este enfoque permite almacenar los datos de manera estructurada, y es fácilmente accesible y editable.

4.1 Funciones Relevantes

4.1.1 Guardar Cliente (`guardar_cliente`)

Esta función permite guardar la información de un cliente en el archivo JSON. Los pasos clave son:

- Se recoge la información del cliente y se organiza en un diccionario.
- Se intenta abrir el archivo `clientes.json` en modo lectura. Si el archivo no existe o está vacío, se inicializa como una lista vacía.
- Se agrega el cliente al listado de clientes.
- Se guarda el listado actualizado en el archivo JSON con formato indentado para mejorar su legibilidad.

4.1.2 Cargar Clientes (`cargar_clientes`)

Esta función carga la información de todos los clientes almacenados en el archivo `clientes.json` y la muestra en una ventana de mensaje. Convierte los diccionarios de clientes a una cadena de texto formateada.

4.1.3 Conversión de Diccionario a String (`_diccionario_a_string`)

Este método estático convierte un diccionario de datos de cliente en un formato de texto legible, lo que es útil para mostrar la información en el GUI.

4.1.4 Cargar Clientes al Sistema (`_cargar_clientes_sistema`)

Este método de clase carga los clientes desde el archivo JSON y los agrega al sistema. Convierte cada cliente almacenado en un objeto `ClienteParticular` y lo agrega al sistema de clientes, cosa clave para la hora de llevar a cabo la primera funcionalidad de nuestra app, que es mostrar los datos de los clientes que están registrados.

4.2 Formato Elegido para Representar Varios Clientes

En el archivo `clientes.json`, los datos de los clientes se almacenan en formato JSON de la siguiente manera:

```
[
    {
        "ID": 1,
        "Nombre": "Juan",
        "Apellidos": "Pérez",
        "Direccion": "Calle Falsa 123",
        "DNI": "12345678A"
    },
    {
        "ID": 2,
        "Nombre": "María",
        "Apellidos": "Gómez",
        "Direccion": "Avenida Siempre Viva 456",
        "DNI": "87654321B"
    }
]
```

Este formato permite almacenar la información de varios clientes como una lista de diccionarios. Cada diccionario representa un cliente con sus respectivos datos (ID, nombre, apellidos, dirección, y DNI).

4.3 Conclusión

La gestión de entrada/salida en este proyecto se maneja mediante archivos JSON para almacenar y recuperar información de clientes. Se emplea un enfoque basado en funciones como `guardar_cliente`, `cargar_clientes`, y `_cargar_clientes_sistema`, las cuales permiten tanto la escritura como la lectura de los datos de manera estructurada.

5 Interfaz Gráfica

La interfaz gráfica del proyecto ha sido diseñada utilizando la librería `tkinter`, una de las herramientas más comunes para la creación de interfaces de usuario en Python. La ventana principal permite gestionar los datos de los clientes, crear eventos, y realizar diversas acciones relacionadas con el sistema de tickets. A continuación, se detalla cómo se ha estructurado la ventana principal y qué funcionalidades permite.

5.1 Planteamiento General

La ventana principal de la aplicación está diseñada para ser clara y funcional, con una disposición sencilla que facilita la interacción con el usuario. La estructura está dividida en tres columnas para organizar los distintos tipos de entradas y botones, y tiene un fondo de color claro que contribuye a la legibilidad y la comodidad visual.

- **Título:** La ventana tiene el título `"Umu Tickets"` y un tamaño de 800x600 píxeles.
- **Fondo:** Se ha elegido un fondo claro (#f9f9f9) para la ventana, que ayuda a crear una interfaz amigable y moderna.
- **Columnas:** La ventana está dividida en tres columnas. La primera y tercera columnas son más estrechas y están destinadas a los botones y las etiquetas. La columna central, más ancha, alberga los campos de entrada de texto.

5.2 Campos de Entrada de Datos

Para registrar un cliente, se proporcionan varios campos de entrada:

- **Identificador:** Este campo se muestra como un `Entry` que es de solo lectura y contiene el ID único del cliente.
- **Nombre, Apellidos, Dirección y DNI:** Para capturar los datos personales del cliente, se dispone de campos de entrada de texto (`Entry`) en los cuales el usuario puede escribir la información.
- **Fuentes y Estilo:** Se utiliza la fuente `Calibri` de 12 puntos para mantener un diseño moderno y legible. Las etiquetas de texto también utilizan esta fuente y se configuran con un color oscuro (#333333) sobre el fondo claro.

5.3 Botones de Acción

Para permitir la interacción del usuario, se han añadido varios botones:

- **Guardar Cliente:** Este botón permite guardar la información introducida en los campos de texto.
- **Cargar Clientes:** Con este botón, el usuario puede cargar la lista de clientes guardados y verlos en una ventana emergente.
- **Simulación:** Este botón permite ejecutar una simulación, lo que puede incluir diferentes funcionalidades como la verificación de la disponibilidad de entradas o la simulación de la compra de tickets. Además, para solucionar el hecho de que en un mensaje no se viera toda la simulación al completo debido a su extensión, hemos decidido mostrar 2 messagebox englobando en cada uno la mitad de la simulación. Así, nos hemos ahorrado el problema anterior y se ve mejor visualmente ya que tambien se planteó realizar una barra de desplazamineto pero se veía menos moderna.

Los botones están diseñados para tener un estilo visual moderno, utilizando colores llamativos como el #E3F2FD para el botón de carga de clientes y el #FFEBEE para el botón de simulación. Cada uno de ellos tiene un efecto al ser presionado, que cambia el color de fondo a tonos más claros (#BBDEFB, #FFCDD2) para dar retroalimentación visual al usuario.

5.4 Disposición de la Ventana

Los elementos de la interfaz se organizan en una estructura de filas y columnas utilizando el método `grid`, lo que facilita el posicionamiento relativo de cada componente en la ventana. A continuación, se describe cómo están dispuestos los componentes:

- Los campos de entrada de datos (como el nombre, los apellidos, la dirección, y el DNI) están colocados en filas consecutivas.
- Los botones de acción (`Guardar Cliente`, `Cargar Clientes`, `Simulación`) están colocados en la fila 6, con una distribución horizontal, ocupando toda la anchura disponible de las columnas.
- Las etiquetas informativas y los campos de entrada de datos se alinean de forma que las etiquetas están a la izquierda y los campos de entrada a la derecha.

5.5 Nota

Los colores y estilos de diseño, como los valores hexadecimales #f9f9f9, #E3F2FD, #BBDEFB, y #FFCDD2, han sido elegidos para ofrecer una interfaz clara y atractiva. Estos tonos suaves fueron seleccionados para crear una experiencia visual cómoda. La información sobre estos colores se ha obtenido en la web **https://www.colorhexa.com/**.

5.6 Capturas de Pantalla de la Ventana en Funcionamiento

A continuación, se incluyen capturas de pantalla de la ventana principal de la aplicación, mostrando el diseño en funcionamiento. **La primera** imagen se corresponde a lo que vería el usuario nada mas entrar, **la segunda** un posible error que puede haber si se intenta reservar más entradas de las que quedan en taquilla, y **la última** es la simulación que tiene que ocurrir:

Introduzca sus datos personales para ser dado de alta.

Identificador: 3777

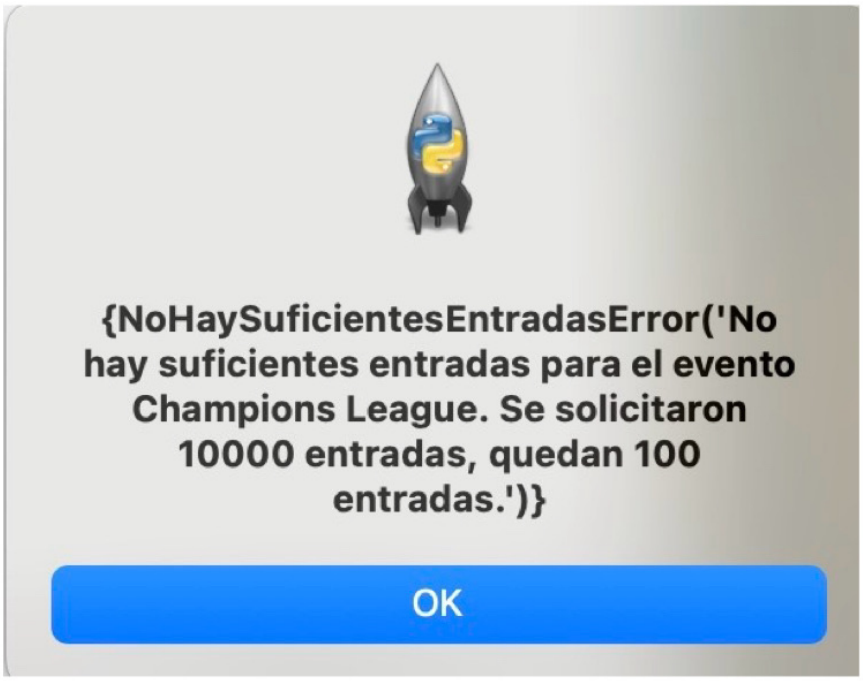
Nombre:

Apellidos:

Direccion postal:

DNI:

Guardar Cliente | Cargar Clientes | Simulación



Simulación completada con éxito!

Clientes actuales en UMUTickets:
Nombre:Francisco , Apellidos:Pellicer Alarcón, Dni:49444970F, Direccion postal: 3001
Nombre:Nina, Apellidos:Esquinas, Dni:4829273C, Direccion postal: 30009

Nuevo cliente (no socio) creado: Paco Pérez García

Nueva clienta (socia) creada: Ana Gómez Ruiz

Nueva empresa creada: TechCorp

Nueva empresa organizadora creada: Wevents S.L

Evento deportivo creado: Champions League
Espectáculo audiovisual creado: Cruz Cafuné en concierto

Evento empresarial creado por la empresa organizadora: Wevents S.L

El cliente no socio ha realizado las siguientes reservas:
Champions League
Cruz Cafuné en concierto

OK

El cliente socio ha realizado las siguientes reservas:
Champions League
Cruz Cafuné en concierto

El cliente empresarial ha realizado la reserva al evento ferial:
Feria de Innovación

Se han pagado los siguientes eventos:
- Cliente socio: Cruz Cafuné en concierto
- Cliente no socio: Champions League
- Cliente empresarial: Feria de Innovación

Intentando anular la reserva del cliente socio al concierto de Cruz Cafuné (ya pagada):
Error: Su reserva al evento Cruz Cafuné en concierto ya ha sido pagada y no se puede anular
Intentando anular la reserva del cliente socio al partido de Champions (no pagada):
La reserva de Champions League ha sido cancelada con éxito.

Se ha realizado la anulación del evento: Champions League, por consiguiente, se les abonará el importe a aquellos que lo hayan pagado.

Las reservas han generado un beneficio de 20.43 euros.

El número de socios actuales es 2 y los ingresos generados por cuotas son de 40 euros.

OK

6 Programa Principal

El archivo `main.py` es el encargado de ejecutar el programa. La estructura principal del archivo se basa en la creación de una ventana gráfica mediante `tkinter`, que es la interfaz de usuario para interactuar con el sistema.

6.1 Estructura del Programa

1. **Inicialización de la ventana**: El programa comienza con la creación de un objeto `Tk()` de `tkinter`, que representa la ventana principal de la interfaz. Luego, se instancia la clase `UmuTicketsGui` pasando el objeto `root` (la ventana principal) como parámetro.

```
if __name__ == "__main__":
    root = tk.Tk()
    app = UmuTicketsGui(root)
    app.muestra_ventana()
```

En este código, el objeto `app` representa la instancia de la interfaz gráfica y llama al método `muestra_ventana()` para mostrar la ventana con los elementos gráficos definidos en la clase `UmuTicketsGui`.

2. **Función de Simulación (`mostrar_simulacion`)**: En la clase `UmuTicketsGui`, el botón "Simulación" activa el método `mostrar_simulacion`, que ejecuta una serie de acciones relacionadas con la simulación de eventos y clientes dentro del sistema.

- **Cargar clientes**: Primero, el método carga los clientes desde un archivo JSON usando la función `cargar_clientes_sistema()` y muestra información sobre los clientes actuales.
- **Alta de clientes**: Luego, se crean y registran nuevos clientes (tanto particulares como empresariales) en el sistema.
- **Creación de eventos**: El sistema crea eventos deportivos y espectáculos audiovisuales, los cuales se asocian con clientes para que realicen reservas.
- **Reservas y pagos**: Los clientes realizan reservas a los eventos y se simula el pago de dichas reservas.
- **Anulación de reservas y eventos**: Se intenta anular algunas reservas y eventos mostrándose los resultados de estas acciones.
- **Beneficios y socios**: Finalmente, el sistema calcula los beneficios generados por las reservas y muestra información sobre el número de socios y los ingresos por sus cuotas.

Todo esto se muestra en un cuadro de mensaje a través de `messagebox.showinfo()`.

6.2 Flujo de Información entre Objetos

El flujo de información entre los objetos es el siguiente:

- `UmuTicketsGui` interactúa con la clase `UMUTickets` para gestionar los clientes, eventos y reservas.
- Se generan mensajes entre la interfaz gráfica y los objetos del sistema (como clientes y eventos), que luego se muestran al usuario.